

Explicit parallel programming in C language is done by using specific libraries and compilers.

In the course we have been exposed to 4 different libraries:

1. **Message Passing Interface**, or MPI. (Library)
2. Posix threads, or Pthreads. (Library)
3. **OpenMP**. (Library and Compiler)
4. **CUDA**. (Library and Compiler)

Usiamo diverse librerie in base alle loro **proprietà**.

Tipi di sistemi paralleli

Un sistema parallelo si suddivide in base alla gestione della memoria dei vari core:

- Memoria **condivisa**.
- Memoria **distribuita**.

Un sistema parallelo viene classificato anche in base alla gestione delle istruzioni date e delle unità di controllo:

- Multiple-Instruction Multiple-Data. (MIMD)
- Single-Instruction Multiple-Data. (SIMD)

	Shared Memory	Distributed Memory
SIMD	CUDA	
MIMD	Pthreads, OpenMP, CUDA	MPI

Sistemi paralleli: A memoria condivisa

Nei sistemi paralleli a memoria condivisa:

- Ogni core ha accesso ad una singola memoria condivisa.
- I core si devono coordinare per evitare *race conditions*, *false sharing*,...
- Dato che condividono lo stesso spazio di memoria, questi sistemi offrono bassa

latenza di comunicazione e usano meno memoria.

- La comunicazione avviene implicitamente.

Sistemi paralleli: A memoria distribuita

Nei sistemi paralleli a memoria distribuita:

- Ogni core ha accesso ad una sua memoria privata.
- I core si devono coordinare tramite l'uso di **messaggi** in una rete.
- La coordinazione quindi avviene esplicitamente.
- Data la distribuzione della memoria, questi sistemi hanno **scalabilità virtualmente infinita**.
- L'invio e ricezione di messaggi introduce un **overhead**.
- È comune anche un overhead di memoria, causata dalla duplicazione di dati spesso richiesti da diversi core.

Sistemi paralleli: Con multiple unità di controllo (MIMD)

Nei sistemi paralleli con multiple unità di controllo:

- Ogni core possiede la sua propria unità di controllo.
- Quindi ogni core può eseguire istruzioni completamente differenti.
- Diversi rami di codice possono essere eseguiti allo stesso tempo.

Sistemi paralleli: Con singola unità di controllo (SIMD)

Nei sistemi paralleli che usano una singola unità di controllo per multipli core:

- Tutti i core di una unità di controllo eseguono la stessa istruzione data, o restano inattivi.
- Ciò può causare lo **stallo** di alcuni core durante l'esecuzione di un ramo di codice, in quanto il sistema non può eseguire entrambi i rami simultaneamente.
- Il sistema quindi eseguirà tali rami in modo **sequenziale**, fermando prima un cammino e poi l'altro, finché non si ricongiungono.

[!Note] Architettura di von Neumann La CPU può leggere e scrivere dati nella memoria.

Il collo di bottiglia di von Neumann si riferisce alla separazione della CPU dalla memoria.

La velocità di trasmissione dei dati dalla memoria a CPU dipende dalla interconnessione usata.

MPI

MPI è una libreria usata per parallelizzare programmi tramite l'invio e ricezione di messaggi basata sul modello di programmazione SPMD.

MPI: Come iniziare e concludere MPI

All'interno di un programma, per usare MPI serve chiamare due funzioni:

- MPI_Init.
- MPI_Finalize.

Prima di chiamare MPI_Init non si può usare la libreria, e dopo aver chiamato MPI_Finalize stessa cosa.

MPI_Init

Serve a effettuare il setup necessario per poter usare la libreria.

MPI_Finalize

Serve ad annunciare che il programma ha concluso, richiedendo a MPI di pulire la memoria allocata per il programma.

MPI: Identificare i processi MPI di un programma

I vari processi MPI di un programma sono identificati dai loro **rank**, un intero positivo.

MPI: Comunicatori

Un comunicatore è un'insieme di processi MPI che possono mandarsi messaggi tra di loro.

La funzione `MPI_Init` quando eseguita, definisce un comunicatore che include tutti i processi MPI creati quando il programma inizia, chiamato `MPI_COMM_WORLD`.

MPI: Comunicatori custom

MPI inoltre fornisce funzioni che servono a creare nuovi comunicatori, questi possono essere utili al programmatore per implementare funzionalità più complesse.

Supponendo di dover utilizzare 2 librerie MPI indipendenti:

- Se si utilizza il comunicatore default (`MPI_COMM_WORLD`) i messaggi potrebbero collidere.
- Si potrebbero assegnare dei **tag** (interi positivi) ai messaggi per distinguerli.
- Sarebbe meglio creare due nuovi comunicatori, uno per libreria.

MPI_Comm_size

Serve a sapere il numero di processi MPI all'interno del comunicatore dato alla funzione come argomento.

MPI_Comm_rank

Serve a sapere il rank del processo MPI chiamante all'interno del comunicatore dato alla funzione come argomento.

MPI: Input al programma

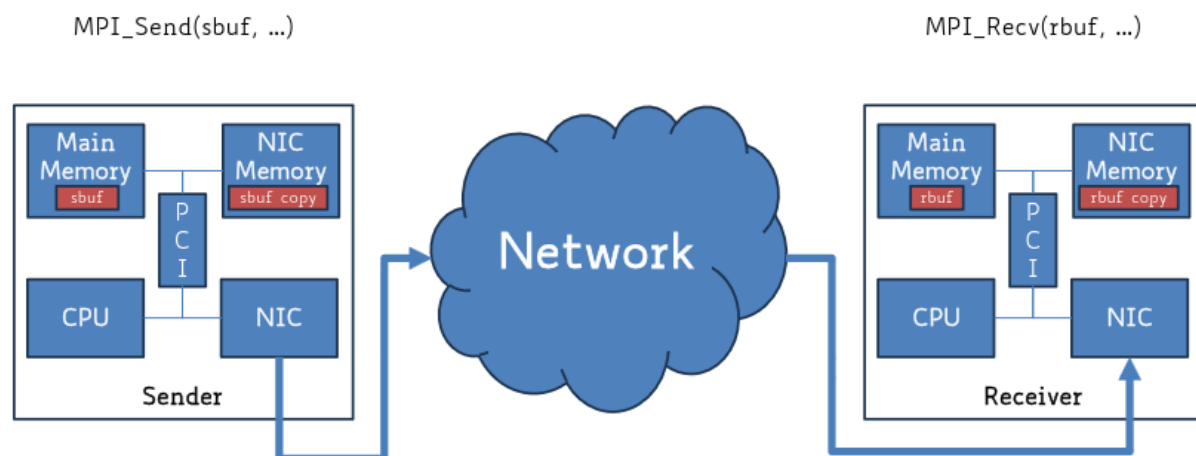
La maggiorparte delle implementazioni di MPI permette solamente al processo 0 del comunicatore `MPI_COMM_WORLD` di accedere allo *stdin*.

Quindi il processo 0 necessariamente deve leggere i dati e mandarli agli altri processi.

MPI: Comunicazione tramite messaggi

La comunicazione tra i vari processi avviene tramite l'invio e ricezione di messaggi.

L'uso di messaggi comporta un overhead importante, quindi è consigliato inviare una sola volta più dati che inviare tante volte piccole porzioni di dati.



MPI_Send

Serve per inviare un messaggio.

```
int MPI_Send( void* msg_buf_p, int msg_size, MPI_Datatype msg_type, int dest, int tag, MPI_Comm comm);
// msg_buf_p è il buffer dove è contenuto il dato da inviare
// msg_type è il tipo di dato del dato da inviare
// dest è il rank del processo alla quale mandare il dato
// tag è usato per differenziare i messaggi inviati da uno stesso processo (serve al destinatario)
```

MPI_Recv

Serve per ricevere un messaggio.

```
int MPI_Recv( void* msg_buf_p, int buf_size, MPI_Datatype buf_type, int source, int tag, MPI_Comm comm, MPI_Status* status_p);
// msg_buf_p è il buffer dove si mette il dato ricevuto
// msg_type è il tipo di dato del dato da ricevere
// source è il rank del processo dalla quale ricevere un dato
// tag è usato per differenziare i messaggi inviati da uno stesso processo
// status_p serve per dare informazioni su una comunicazione
```

MPI: Messaggi non sorpassanti

I messaggi tra due processi necessariamente devono essere **non sorpassanti**:

- Se un processo invia più messaggi allo stesso processo di destinazione, tali messaggi devono essere disponibili al destinatario nella stessa sequenza in cui sono stati inviati. Mentre per i messaggi inviati da processi differenti non si applica tale regola.

MPI: Ricezione corretta dei messaggi

Un messaggio viene ricevuto **correttamente** se:

- Il tipo del dato scelto durante la ricezione corrisponde a quello indicato durante l'invio.
- E se la dimensione del buffer dove mettere il dato ricevuto è sufficiente.

Un processo può ricevere messaggi anche se:

- Non conosce la dimensione del dato ricevuto, tramite la funzione `MPI_Get_count`.
- Non conosce chi lo manda, tramite `MPI_ANY_SOURCE`.
- Non conosce il tag del messaggio, tramite `MPI_ANY_TAG`.

MPI: Ottenere il tag o rank di un messaggio ricevuto tramite ANY_SOURCE o ANY_TAG

Serve a **recuperare** il rank o il tag **mancante** tramite `MPI_Status` quando l'operazione di ricezione (come `MPI_Recv`) viene utilizzato con `MPI_ANY_SOURCE` oppure `MPI_ANY_TAG`.

Questo perché usando questi argomenti per un'operazione di ricezione, il processo può ricevere correttamente un messaggio senza conoscere il suo mittente o tag.

Poiché queste informazioni vengono omesse durante la chiamata, MPI inserisce queste informazioni all'interno di `MPI_Status` per fornire i dettagli del messaggio.

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, MPI_ANY_SOURCE, MPI_ANY_TAG, recv_
comm, &status);

int rank_sender = status.MPI_SOURCE;
int tag = status.MPI_TAG;
```

MPI_Get_count

Serve per ottenere la dimensione del dato contenuto nel messaggio da ricevere.

```
int MPI_Get_count( MPI_Status* status_p, MPI_Datatype type, int* count_p);
// Dare in input il tipo del dato e lo status del messaggio ricevuto
// La grandezza del messaggio ricevuto verrà inserito in count_p
```

MPI: Problemi con l'invio e la ricezione di messaggi

Il comportamento esatto delle due funzioni è determinato dalla implementazione di MPI usata, quindi non si deve assumere niente sul loro comportamento fuori da ciò che viene definito dallo standard MPI, altrimenti il codice potrebbe essere poco portatile.

MPI_Send potrebbe comportarsi in modo diverso in base alla:

- Dimensione del buffer.
- Cutoffs.
- Blocking.

MPI_Recv blocca sempre l'esecuzione del processo che lo chiama finché non riceve un messaggio che corrisponde agli argomenti indicati alla chiamata.

MPI: Comunicazione tra più processi contemporaneamente

Quando due o più processi hanno bisogno di scambiarsi dati a vicenda, questi hanno bisogno di effettuare ognuno un'invio e una ricezione, e l'**ordine** con cui le due

operazioni vengono effettuate è importante:

- Se entrambi effettuano una ricezione, allora andranno a bloccare la loro esecuzione in attesa di un messaggio, finendo in **deadlock**.
- Se entrambi effettuano un'invio, è possibile comunque finire in **deadlock** se la dimensione del dato inviato è grande.

Per evitare queste situazioni, è consigliato usare la funzione `MPI_Sendrecv`.

MPI_Sendrecv

Serve per inviare e ricevere simultaneamente un messaggio.

```
int MPI_Sendrecv(void* send_buf_p, int send_buf_size, MPI_Datatype send_buf_type,
int dest, int send_tag,
void* recv_buf_p, int recv_buf_size, MPI_Datatype recv_buf_type, int source, int recv_tag,
MPI_Comm communicator, MPI_Status* status_p);
```

MPI: Tipi di comunicazione

Esistono vari tipi di comunicazione:

- Bufferata (Buffered).
- Sincrona (Synchronous).
- Ready.

MPI: Tipo di comunicazione bufferata

Nella modalità di comunicazione bufferata, le operazioni di invio sono **sempre localmente bloccanti**, cioè sbloccano il processo chiamante non appena il messaggio inviato viene copiato in un buffer. Tale buffer è **fornito dall'utente**.

MPI: Tipo di comunicazione sincrona

Nella modalità di comunicazione sincrona, le operazioni di invio (`MPI_Send`) bloccano il processo mittente.

Lo sblocco avviene solo dopo che il processo destinatario comincia l'operazione di ricezione del messaggio.

Tale operazione è una operazione **globalmente bloccante**, cioè il processo mittente **conosce lo stato** del processo destinatario in quanto il mittente si sblocca quando il destinatario comincia la ricezione del messaggio. Non servono altri messaggi per confermare l'avvenuta ricezione.

MPI: Tipo di comunicazione ready

Nella modalità di comunicazione ready, le operazioni di invio hanno successo solamente se un'operazione di ricezione **corrispondente** è **già iniziata**, altrimenti l'invio restituisce un **codice di errore**.

Questa modalità serve a ridurre l'overhead delle operazioni di handshaking.

MPI: Comunicazione non bloccante

Le operazioni di invio bufferate sono viste negativamente in quanto comportano un calo della performance, causato dal **blocco del processo chiamante**, che deve aspettare che il dato venga copiato nel buffer.

Quindi funzioni non bloccanti o immediate portano a **massimizzare la concorrenza** in quanto:

- Sbloccano il processo chiamante immediatamente non appena viene inizializzato un trasferimento di dati.

In questo modo un processo può eseguire altre operazioni mentre il trasferimento viene effettuato.

Per questo sia MPI_Send che MPI_Recv hanno delle versioni non bloccanti e immediate.

Se i processi utilizzano le comunicazioni non bloccanti, questi necessariamente hanno bisogno di eseguire altre comunicazioni usate per conoscere lo stato di completamento della comunicazione iniziale:

- I processi mittente per poter modificare o riutilizzare il buffer del messaggio.
- I processi destinatario per poter cominciare ad estrarre il contenuto del messaggio.

Le comunicazioni non bloccanti possono essere unite con ogni tipo di comunicazione:

- MPI_Ibsend, MPI_Issend, MPI_Irsend, ... (Buffered, Synchronous, Ready)

MPI: Controllare lo stato di completamento di una comunicazione

Esistono due funzioni principali per controllare lo stato di completamento di una comunicazione:

- MPI_Wait, bloccante.
- MPI_Test, non bloccante.

Altre varianti di queste due esistono, come:

- MPI_Waitall.
- MPI_Testall.
- MPI_Waitany.
- MPI_Testany.
- ...

MPI_Wait

Serve per bloccare un processo che ha un operazione di invio o ricezione in corso.

MPI_Test

Serve a controllare se una operazione di invio o ricezione è stata completata o meno.

MPI: Comunicazioni collettive

Le comunicazioni collettive sono operazioni che coordinano il **trasferimento** o la sincronizzazione di dati tra un **insieme di processi** all'interno di uno specifico comunicatore.

Per poter avviare una funzione collettiva, **tutti** i processi all'interno di uno specifico comunicatore devono chiamare la **stessa** funzione collettiva.

Siccome queste funzioni sono collettive, non richiedono come argomento un **tag**.

Il risultato della funzione collettiva può essere usato solamente dal processo indicato come destinatario della collettiva. Tutti gli altri processi inseriranno *NULL* come argomento del buffer di output.

Inoltre, gli argomenti passati in input da ogni processo alla funzione collettiva devono

essere **compatibili**.

Per esempio:

- Un processo passa in input come destinazione della collettiva il processo con rank 0 e un altro processo passa il processo con rank 1, allora la funzione MPI_Reduce va in errore e il programma molto probabilmente crasha o si blocca.

MPI_Reduce

È una funzione di **riduzione** di dati basata sull'**operatore MPI** dato come argomento.

```
int MPI_Reduce( void* input_data_p, void* output_data_p, int count, MPI_Datatype
e datatype, MPI_Op operator, int dest_process, MPI_Comm comm);

// Esempio di uso
MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

MPI_Reduce Operators

Operation Value	Meaning
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI_LAND	Logical and
MPI_BAND	Bitwise and
MPI_LOR	Logical or
MPI_BOR	Bitwise or
MPI_LXOR	Logical exclusive or
MPI_BXOR	Bitwise exclusive or
MPI_MAXLOC	Maximum and location of maximum
MPI_MINLOC	Minimum and location of minimum

You can create custom operators with **MPI_Op_create**

MPI_Bcast

È una funzione di **broadcasting** che invia a tutti i processi all'interno del comunicatore di appartenenza i dati indicati dal processo chiamante.

```
int MPI_Bcast(void* data_p, int count, MPI_Datatype datatype, int source_proc,
MPI_Comm comm);

// data_p è sia input che output
```

MPI_Allreduce

È una combinazione di due funzioni collettive:

- MPI_Reduce e MPI_Bcast.

Cioè viene eseguito prima la riduzione, seguito da un broadcast del risultato ottenuto dalla riduzione.

```
int MPI_Allreduce(void* input_data_p, void* output_data_p, int count, MPI_Datatype datatype, MPI_Op operator, MPI_Comm comm)
```

MPI_Scatter

Serve per **distribuire** un vettore che viene letto dal processo con rank 0 a tutti gli altri processi, tale operazione manda a ognun processo solamente la **partizione** sulla quale deve operare.

```
int MPI_Scatter(void* send_buf_p, int send_count, MPI_Datatype send_type, void*
recv_buf_p, int recv_count, MPI_Datatype recv_type, int src_proc, MPI_Comm com
m);

// send_count indica il numero di elementi da mandare a ogni processo
// e non indica il numero totale di elementi del vettore

// recv_buf_p = MPI_IN_PLACE -> la partizione assegnata al processo
// chiamante non viene copiata e poi reincollata in un altro buffer
// ma rimane nel buffer iniziale
```

Per mandare numeri differenti di elementi a ogni processo esiste la variante MPI_Scatterv.

MPI_Gather

Serve per **raccogliere tutte** le partizioni del vettore presenti nei vari processi, e inviare il tutto nel buffer del processo con rank 0.

```
int MPI_Gather(void* send_buf_p, int send_count, MPI_Datatype send_type, void*,
recv_buf_p, int recv_count, MPI_Datatype recv_type, int dest_proc, MPI_Comm com
m);

// send_count indica il numero di elementi da mandare a ogni processo
// e non indica il numero totale di elementi del vettore
```

MPI_Allgather

È una combinazione di due funzioni:

- MPI_Gather e MPI_Bcast.

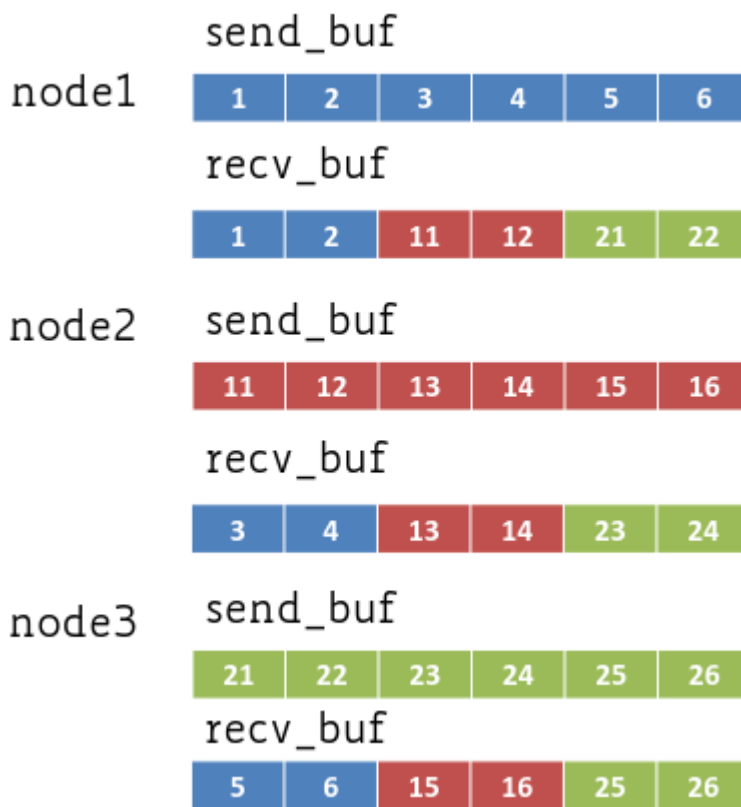
Cioè viene eseguito prima la raccolta dei dati, seguito da un broadcast del vettore ottenuto dalla raccolta.

```
int MPI_Allgather(void* send_buf_p, int send_count, MPI_Datatype send_type, void*
recv_buf_p, int recv_count, MPI_Datatype recv_type, MPI_Comm comm);

// send_count e recv_count indicano il numero di elementi mandati da ogni processo
```

MPI_Alltoall

Serve ad inviare una partizione del proprio vettore a tutti gli altri processi che chiamano la funzione.



MPI: Rilevanza degli algoritmi delle funzioni collettive

Data l'importanza delle funzioni collettive, e il loro maggiore impatto sul tempo di esecuzione totale del programma rispetto alle altre operazioni, queste hanno bisogno di essere ottimizzate per poter decidere quale algoritmo usare in base alla situazione attuale, quanti processi sono coinvolti, ...

Per questa ragione esistono varie implementazioni di librerie di funzioni collettive.

MPI: Tipi di dato derivati

In MPI, **derived datatypes** allow developers to create custom communication structures for complex data that goes beyond basic, predefined types (like standard integers or floats). They solve the problem of needing to send or receive collections of mixed or non-contiguous data items (such as a C `struct`) over the network without requiring manual buffer management.

Here is a breakdown of how they work and why they are useful:

- **Mapping Memory Layout:** A derived datatype is formally defined by specifying a **sequence of basic MPI data types along with their specific memory displacements** (their relative locations or offsets in memory).
- **Automatic Packing on Send:** Instead of forcing the programmer to write manual code to pack different variables into a single, contiguous buffer before sending, you can simply pass the derived datatype to your send function. Because the send function understands the types and exact memory locations of all the items, **it automatically collects the scattered items directly from memory** and packages them for transmission.
- **Automatic Unpacking on Receive:** Similarly, when the destination process receives the message, the receive function uses the derived datatype to **automatically distribute the incoming data items directly into their correct, corresponding memory locations**.

How They Are Created MPI provides several specialized functions to build these custom datatypes depending on how your data is laid out:

- `MPI_Type_create_struct` : Builds a derived datatype consisting of individual elements that have different basic types (perfect for standard C `structs`).
- `MPI_Type_contiguous` : Builds a datatype for a series of contiguous elements.
- `MPI_Type_vector` : Builds a datatype for elements of the same type that are repeated and separated by a regular interval or "stride".
- `MPI_Type_create_subarray` : Used to select and communicate multidimensional subarrays.

When creating these types—especially structs—it is highly recommended to use

`MPI_Get_address` to find the exact memory displacements of the items. This ensures the derived datatype correctly accounts for any invisible memory padding the compiler might introduce, which can vary between 32-bit and 64-bit platforms. Once the datatype is built, you must call `MPI_Type_commit` to allow the MPI library to optimize its internal representation before you can use it in a communication function.

MPI: Valutazione della performance (Profiling)

Durante lo sviluppo del programma parallelo, è possibile controllare il periodo di tempo trascorso tra due punti del codice.

MPI_Wtime

Serve ad ottenere il numero di secondi trascorsi da qualche tempo prima.

```
double MPI_Wtime(void);

// Esempio d'uso
start = MPI_Wtime();
...
finish = MPI_Wtime();
printf("Elapsed time = %e seconds\n", finish-start);
```

MPI: Threading

I processi MPI possono usare **threads** per velocizzare ancora di più un programma.

Per poter usare threads, bisogna chiamare `MPI_Init_thread` al posto di `MPI_Init`, indicando necessariamente il livello di threading richiesto e il livello di threading supportato dal sistema.

MPI: Livello di threading

In MPI esistono 4 livelli di threading:

1. `MPI_THREAD_SINGLE`.
 - Nessun rank può usare threads.

2. MPI_THREAD_FUNNELED.

- Ogni rank può usare threads, ma solo il **thread principale** può chiamare funzioni MPI.
- Ideale per parallelizzazione **fork/join** come in `#pragma omp parallel` dove tutte le chiamate MPI sono fuori dai blocchi OpenMP.

3. MPI_THREAD_SERIALIZED.

- Ogni rank può usare threads, ma solamente un thread alla volta può chiamare funzioni MPI.
- Un thread deve quindi assicurarsi di essere l'unico al momento a chiamare una funzione MPI, realizzabile tramite l'uso di **mutex**.

4. MPI_THREAD_MULTIPLE.

- Ogni rank può usare threads, e non ci sono limitazioni.
- La libreria MPI deve assicurarsi che gli accessi ai dati sono sicuri per tutti i threads.

Valutazione della performance

Rumore (Noise during runs)

Per ridurre il rumore che va ad influire sui dati ottenuti durante le misurazioni, è possibile usare delle **barriere**, che se poste all'inizio del programma, vanno ad assicurare più o meno che tutti i processi inizino allo stesso tempo.

Per questo:

- Più il numero di processi/threads/GPUs aumenta, più è possibile che almeno uno di questi venga influenzato dal rumore.

Proporzione numero di processi : dimensione del problema

Generalmente:

- Il tempo di esecuzione aumenta all'aumentare della dimensione del problema.
- Il tempo di esecuzione diminuisce all'aumentare del numero di processi.

Esiste però un **limite** alla riduzione del tempo di esecuzione all'aumentare del numero

di processi:

- Se il numero di processi X è abbastanza grande da computare il problema.
- Usando $Y \gg X$ processi è possibile che non si abbia abbastanza lavoro da suddividere per tutti i processi, in tal caso alcuni processi resterebbero inutilizzati, non portando quindi a **nessuna riduzione del tempo di esecuzione**.

Riduzione del tempo di esecuzione (Speedup)

Idealmente, quando si esegue un programma con X processi, questo dovrebbe essere X volte più veloce rispetto a quando lo si esegue con 1 processo.

Dato il tempo seriale $T_s(n)$ e il tempo parallelo usando X processi $T_p(n, X)$, l'**aumento di velocità** di esecuzione del programma calcolato usando $S(n, X) = \frac{T_s(n)}{T_p(n, X)}$ idealmente dovrebbe essere uguale a X , risultando in un **aumento lineare**.

Inoltre, il tempo parallelo usando 1 processo, generalmente dovrebbe essere più o veloce uguale **rispetto al tempo sequenziale**.

Scalabilità del programma parallelo (Scalability)

Dato il tempo parallelo usando 1 processo $T_p(n, 1)$ e il tempo parallelo usando X processi $T_p(n, X)$, la **scalabilità**, ovvero quanto il programma parallelo riesce ad utilizzare al meglio più processi per velocizzare la computazione, viene calcolato usando $S(n, X) = \frac{T_p(n, 1)}{T_p(n, X)}$

Efficienza (Efficiency)

Idealmente l'efficienza di un programma parallelo dovrebbe essere uguale ad 1.

Se tale valore è minore di 1, allora il programma **performerà peggio** all'aumentare della dimensione del problema.

L'efficienza viene calcolata usando $E(n, X) = \frac{T_s(n)}{X \times T_p(n, X)}$

Strong Scaling e Weak Scaling

Lo **strong scaling** è calcolabile:

- Fissando la dimensione del problema, si va ad aumentare il numero di processi.
- E se l'efficienza rimane alta, allora il programma parallelo è **strong scalable**.

Il **weak scaling** è calcolabile:

- Aumentando la dimensione del problema allo **stesso ritmo** con cui si aumenta il numero di processi.
- Quindi se si aumenta di un fattore 2 la dimensione del problema, si aumenta di un fattore 2 il numero di processi.
- E se l'efficienza rimane alta, allora il programma parallelo è **weak scalable**.

Legge di Amdahl

Ogni programma ha delle parti che sono **impossibili da parallelizzare**:

- Leggere e scrivere sul disco, mandare e ricevere dati dalla rete, ...

La legge di Amdahl enuncia che lo **speedup** è **limitato** dalla frazione seriale (**serial fraction**), cioè $T_p(X) = (1 - \alpha) \times T_s + \alpha \times \frac{T_s}{X}$ dove $1 - \alpha$ è la parte di codice eseguibile solo sequenzialmente.

Legge di Amdahl: Limitazioni

La frazione seriale potrebbe aumentare quando aumentano il numero di **processori**. (?)

Legge di Gustafson

Se si considera il **weak scaling**, la frazione parallela (**parallel fraction**) aumenta all'aumentare della dimensione del problema.

Ciò è anche noto come **scaled speedup**, ovvero $S(n, X) = (1 - \alpha) + \alpha \times X$

Progettazione (Design) di programmi paralleli (Metodologia di Foster)

La metodologia di Foster descrive i procedimenti da seguire per progettare programmi paralleli.

Fornisce un approccio per scomporre il problema da computare in modo da rendere

l'esecuzione parallela efficiente.

Si divide in 4 fasi:

- Partizionamento (Partitioning).
- Comunicazione (Communication), a seguito del partizionamento.
- Aggregazione (Agglomeration).
- Mappatura (Mapping)

Il partizionamento serve per dividere la computazione da eseguire e i dati su cui si opera in parti più piccole. L'obiettivo principale è identificare quali operazioni possono essere eseguiti in parallelo.

La comunicazione serve per determinare la comunicazione necessaria da usare per riuscire ad eseguire le operazioni definite durante il partizionamento.

L'aggregazione serve per combinare operazioni legate tra di loro in un'unica operazione più grande (Dipendenza). Per esempio se un'operazione A necessariamente viene prima di B, allora ha senso combinarli. (Per avere meno overhead)

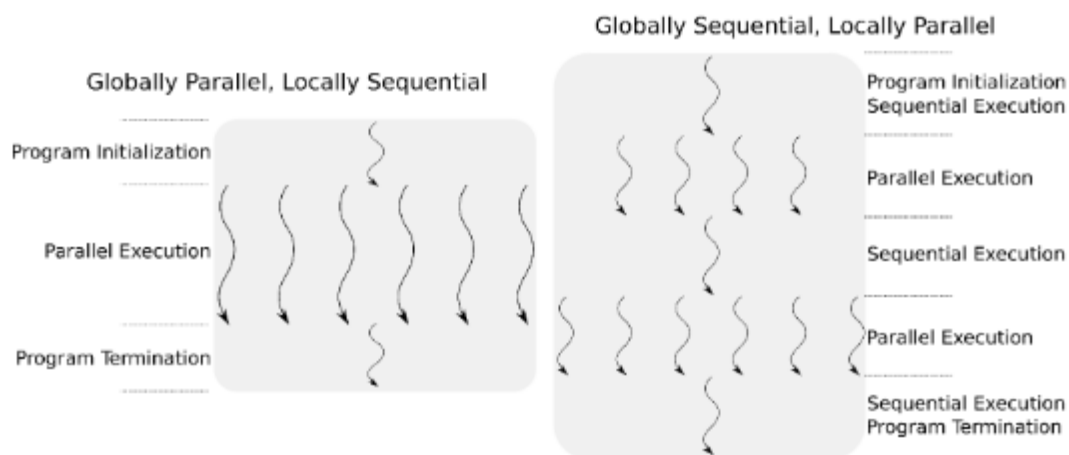
La mappatura serve per assegnare le operazioni definite nei passi precedenti, in modo da minimizzare la comunicazione necessaria e assicurarsi che ogni processo o thread ottenga la stessa quantità di operazioni da eseguire.

Modelli (Patterns) di progettazione parallela

I modelli di progettazione parallela servono ad organizzare i programmi paralleli, e si differenziano in base a come gestiscono il flusso dell'esecuzione del programma.

Tutti i modelli di progettazione parallela rientrano in 2 categorie:

- Globally Parallel, Locally Sequential (GPLS).
- Globally Sequential, Locally Parallel (GSLP).



Modello di progettazione parallela: Globalmente parallelo e localmente sequenziale (GPLS)

I programmi che si basano su questo modello sono progettati per eseguire più operazioni contemporaneamente, con ogni singola operazione che esegue le proprie istruzioni in modo sequenziale.

Modelli che rientrano in questa categoria:

- Single-Program, Multiple-Data (SPMD).
- Multiple-Program, Multiple-Data (MPMD).
- Master-Worker.
- Map-Reduce.

Single-Program Multiple-Data (SPMD)

Nel modello di programmazione SPMD, si compila un **singolo** programma contenente **tutta la logica** dell'applicazione che viene eseguito da multipli processi.

Tali processi vengono controllati tramite l'uso di blocchi *if-else* e hanno bisogno di comunicare tramite l'invio e ricezione di messaggi in quanto non condividono la memoria.

Questo modello **fallisce** quando:

- I requisiti di memoria sono troppo alti per ogni nodo.

- Oppure quando sono coinvolte piattaforme eterogenee. (?)

Multiple-Program Multiple-Data (MPMD)

È simile a SPMD, ma esegue l'avvio di multipli programmi diversi che lavorano insieme.

Master-Worker

In questo modello i processi si differenziano in Masters e Workers.

I Masters sono responsabili di:

- Distribuire parti di lavoro da far eseguire ai workers.
- Collezionare i risultati ottenuti dai workers.
- Eseguire operazioni di I/O per conto dei workers.
- Interagire con l'utente.

Il vantaggio di questo modello è che il carico di lavoro viene bilanciato **implicitamente**, anche se il processo Master può diventare un punto di **bottleneck**.

Tale collo di bottiglia è riducibile tramite l'uso di una **gerarchia** di Masters.

Map-Reduce

È una variante del modello Master-Worker.

I Masters gestiscono le operazioni da eseguire mentre i workers possono eseguire due tipi di operazioni:

- Map, cioè applicare una funzione a sezioni di un dato per generare risultati parziali.
- Reduce, cioè collezionare e combinare i risultati parziali per ottenere il risultato finale.

Modello di progettazione parallela: Globalmente sequenziale e localmente parallelo (GSLP)

I programmi che si basano su questo modello sono progettati per eseguire un singolo processo inizialmente, e da questo si **generano altri processi** per eseguire alcune parti in modo parallelo quando richiesto.

Modelli che rientrano in questa categoria:

- Fork/Join.
- Loop parallelism.

Fork/Join

In questo modello l'esecuzione inizia con un singolo thread radice, per poi generare tramite **fork** thread figli che vanno ad eseguire le operazioni assegnate.

Il thread padre **non** può continuare la sua esecuzione finché tutti i figli non finiscono le loro operazioni e ritornano al padre tramite **join**.

Questo è il modello usato da OpenMP e Pthread.

Loop parallelism

Questo modello viene utilizzato per effettuare **migrazione** di software **legacy** o sequenziali.

Si concentra sullo scomporre i cicli tramite **manipolazione della variabile di controllo del ciclo**, in modo che thread diversi eseguano iterazioni del ciclo diverse.

Da notare però che **non** tutti i cicli sono supportati, in quanto devono avere una forma molto specifica e **prevedibile**.

OpenMP (Open MultiProcessing)

OpenMP è una libreria usata per parallelizzare programmi in sistemi a memoria condivisa, e tali programmi seguono il modello Globally-Sequential Locally-Parallel (GSLP), in particolare tramite **fork/join**.

Tale sistema è visto come un'insieme di core o CPU, le quali hanno tutti accesso ad una memoria principale **condivisa**.

OpenMP si affida a delle **direttive del compilatore** per segnalare i blocchi di codice che il compilatore proverà a parallelizzare.

OpenMP: Gestire il numero di thread da usare

Esistono 3 livelli di controllo del numero di thread da usare all'interno di un processo:

- **Universale**, definendo il numero di threads tramite una **variabile d'ambiente**.

- Livello del **programma**, definendo il numero di threads chiamando la funzione `omp_set_num_threads` al di fuori dai blocchi di codice segnati dalle direttive OpenMP.
- Livello del **pragma**, definendo il numero di thread nella stessa riga della direttiva pragma OpenMP usando la clausola `num_threads`.

La **precedenza** viene data prima al numero di thread indicato più specificatamente, ovvero:

- **Più importante:** livello del pragma.
- **In mezzo:** livello del programma.
- **Meno importante:** Universale.

OpenMP: Team

In OpenMP, l'insieme del thread originale più i nuovi threads generati che eseguono un blocco di codice parallelo viene definito come **team**.

(Similmente ai comunicatori in MPI)

OpenMP: `omp_get_num_threads` (funzione)

Serve ad ottenere il numero di threads attivi in un blocco di codice parallelo.

Se la funzione viene chiamata in una regione sequenziale, restituisce 1.

OpenMP: `omp_get_thread_num` (funzione)

Serve ad ottenere il rank (id) del thread chiamante.

OpenMP: Pragmas

OpenMP si affida come detto alle delle direttive del compilatore, in particolare usa `#pragma` per indicare al compilatore dei comportamenti specifici.

OpenMP: `pragma omp parallel` (direttiva)

È la direttiva di parallelizzazione più semplice.

Il numero di threads che eseguono il blocco di codice delimitato viene **determinato dal**

sistema a tempo di esecuzione.

```
# pragma omp parallel num_threads(thread_count)
{
    function();
}

// int my_rank = omp_get_thread_num(); // Rank del thread chiamante
// int thread_count = omp_get_num_threads(); // Numero totale di threads
```

Dopo l'esecuzione di un blocco di codice parallelo, una **barriera implicita** bloccherà i vari thread.

OpenMP: pragma omp end parallel (direttiva)

Serve per indicare la fine di un blocco di codice parallelo.

OpenMP: sezioni critiche tramite clausola critical (clausola)

Serve per indicare un **blocco di codice** che può essere eseguito da un solo thread alla volta.

```
# pragma omp critical
{
    global_result += my_result;
}
```

OpenMP: sezioni critiche denominate

OpenMP fornisce l'opzione di **aggiungere un nome** alle direttive `critical`.

È possibile eseguire direttive `critical` con **nomi diversi simultaneamente**, quindi più di un thread alla volta può eseguire lo stesso blocco di codice critico.

Per poter fare ciò, serve definire queste direttive a **tempo di compilazione**.

```
# pragma omp critical(name)
{
    // critical section
}
```

OpenMP: Locks

OpenMP fornisce una sua implementazione delle chiavi (locks).

```
omp_lock_t writelock; // dichiarazione
omp_init_lock(&writelock); // inizializzazione

# pragma omp parallel for
{
    for (i = 0; i < x; i++) {
        ...
        omp_set_lock(&writelock); // ottieni la chiave
        ... // Codice eseguibile solo da un thread alla volta
        omp_unset_lock(&writelock); // restituisci la chiave
        ...
    }
}
```

OpenMP: pragma omp atomic (clausola)

Serve per indicare che un'**operazione sulla memoria** deve essere effettuata in modo **atomico**, ovvero senza interferenze da altri thread.

La clausola `atomic` è più **efficiente e veloce** di `critical` in quanto deve proteggere solamente un'operazione invece che un blocco di operazioni.

La variabile viene protetta in lettura, scrittura e modifica, ma la parte destra dell'operazione viene **comunque eseguito in parallelo**.

```
# pragma omp atomic
    global_result += function();

// global_result è protetto
// function() viene comunque eseguito da tutti i thread
```

OpenMP: Cosa usare tra sezioni critiche, operazioni atomiche e locks

Generalmente le direttive atomiche in teoria sono il metodo più veloce per ottenere la **mutua esclusione**, tuttavia in base all'implementazione OpenMP scelta:

- Le direttive atomiche potrebbero applicare la mutua esclusione attraverso **tutte** le direttive atomiche **presenti nell'intero programma**.
- Quindi anche operazioni atomiche che **non** si influenzano minimamente potrebbero essere mutualmente escluse.
- (Tutto ciò varia in base all'implementazione scelta di OpenMP)

L'uso delle chiavi (locks) invece è consigliato da usare per situazioni dove serve la mutua esclusione per una **struttura dati** piuttosto che per un blocco di codice.

È **pericoloso** annidare (nest) costrutti usati per implementare la mutua esclusione.

Infine, è **sconsigliato** e anche **pericoloso** mischiare costrutti differenti per implementare la mutua esclusione, in quanto non c'è una **garanzia** su chi viene eseguito prima (**fairness**).

OpenMP: pragma omp parallel reduction (clausola)

In OpenMP, una riduzione serve a computare un dato, ottenuto applicando lo stesso operatore di riduzione ad una sequenza di operandi.

Tutti i risultati intermedi ottenuti applicando l'operatore durante l'operazione di riduzione sono salvati in una stessa variabile.

Un'operatore di riduzione è un'operazione binaria:

- Addizione o sottrazione `+, -`.
- Moltiplicazione o potenza `*, ^`.
- And o Or `&&, ||`.
- Bitwise And o Or `&, |`.

Ogni thread al suo interno ha una copia privata della variabile indicata dalla clausola `reduction( operatore : variable)`, e tale variabile viene inizializzata al **valore d'identità** dell'operatore usato.

Alla fine del blocco di codice parallelo, avviene la riduzione, che va ad accumulare i valori computati all'interno del blocco insieme al valore presente fuori dal blocco.

```
int var;
# pragma omp parallel num_threads(thread_count) reduction(* : global_result)
{
    var += function(); // Qui global_result è inizializzato a 1 (valore d'identità per moltiplicazioni)
}

// var = var * var_di_thread_1 * ... * var_di_thread_thread_count
```

OpenMP: Scope delle variabili

In OpenMP, per scope di una variabile si intende l'insieme dei threads che possono accedere a tale variabile all'interno di un blocco di codice parallelo.

Una variabile che può essere acceduta da tutti i thread di un team:

- Ha scope **condivisa** (shared).

Una variabile che può essere acceduta solamente da un singolo thread:

- Ha scope **privata** (private).

Di default ogni variabile **dichiarata al di fuori** di un qualsiasi blocco OpenMP:

- Ha scope condivisa.

OpenMP: Specificare lo scope delle variabili all'interno di un blocco (clausola)

È possibile usare la clausola `default` per manualmente specificare lo scope di ogni variabile che viene usata in un blocco di codice parallelo.

Lo scope predefinito delle variabili **dichiarate al di fuori** dei blocchi di codice parallelo è `shared`.

La clausola `shared` bisogna esplicitamente indicarla quando `default(none)` viene specificato.

OpenMP: private (clausola)

La clausola `private` se indicata, va a creare **una copia separata** di una variabile **per ogni** thread nel team. Queste copie **non** sono inizializzate, quindi all'interno del blocco di codice parallelo **non hanno un valore assegnato**.

```
int shared_var = 10;
# pragma omp parallel default(none) shared(shared_var) private(private_var)
{
    int private_var = 1; // Serve inizializzare private_var
    private_var += shared_var;
}
```

OpenMP: firstprivate (clausola)

La clausola `firstprivate` è una versione della clausola `private` più avanzata, in quanto la variabile indicata dalla clausola viene **automaticamente inizializzata** all'interno di ogni thread del team con il valore che ha la stessa variabile ma fuori dal blocco di codice parallelo.

```
int private_var = 1;
# pragma omp parallel firstprivate(private_var)
{
    // Non serve inizializzare private_var
    private_var += 10; // 1 + 10
}
```

OpenMP: lastprivate (clausola)

La clausola `lastprivate` è una versione della clausola `private` più avanzata, in quanto la variabile indicata dalla clausola viene **automaticamente trasferita fuori** alla fine della computazione del blocco di codice parallelo dall'ultimo thread che lo esegue.

Inoltre è possibile usare `firstprivate` e `lastprivate` insieme.

```
int private_var = 1;
# pragma omp parallel firstprivate(private_var) lastprivate(private_var)
{
    private_var += 10; // 1 + 10
}
printf(private_var); // 11
```

OpenMP: threadprivate e copyin (clausole)

Persistent Global Variables (`threadprivate` and `copyin`) Sometimes, a thread needs a private variable that doesn't just disappear when a single parallel loop ends, but rather lives on to be used in future parallel regions.

- `threadprivate` : This directive is used to make **global or static variables** private to each thread. Unlike standard private variables, **the value of a `threadprivate` variable is preserved and persists across multiple different parallel regions** throughout the entire execution of the program. This allows each thread to maintain its own independent state or memory across distinct phases of your computation.

- `copyin` : Because `threadprivate` variables are persistent across the whole program, they do not get automatically initialized when a new parallel region begins. If you want all threads to start a new parallel region with the same baseline value for a `threadprivate` variable, you use the `copyin` clause. It **takes the value currently held by the master thread and copies it into the `threadprivate` variables of all the other threads in the team.**

OpenMP: pragma omp parallel for (direttiva)

Serve a far eseguire un ciclo `for` da un team di threads.

Il blocco di codice segnato da questa direttiva necessariamente **deve essere** un ciclo `for` .

La parallelizzazione del ciclo viene eseguito tramite l'**assegnamento delle iterazioni** del ciclo tra i vari threads.

Inoltre OpenMP impone la dichiarazione del ciclo in un modo specifico, in quanto serve al sistema per determinare il numero di iterazioni a tempo di esecuzione.

for (	index < end	index++
	index <= end	++index
	index >= end	index--
	index > end	--index
	index = start ;	index += incr
	index >= end ;	index -= incr
	index > end	index = index + incr
		index = incr + index
	index = index - incr	

OpenMP: limitazioni sulla forma della dichiarazione del ciclo

La variabile `index` deve essere di tipo `integer` o puntatore a `integer` , o sue versioni più grandi come `long` .

Le espressioni `start` , `end` e `incr` devono essere compatibili con `index` .

Le espressioni `start` , `end` e `incr` **non** devono cambiare durante l'esecuzione del ciclo.

Durante l'esecuzione del ciclo, la variabile `index` può solamente essere modificata dall'espressione `incr` definito nella dichiarazione del ciclo.

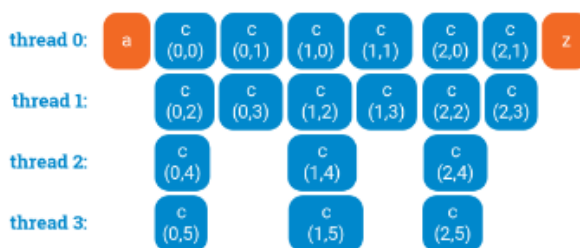
Non è possibile inserire `break` o `return` all'interno del ciclo.

OpenMP: cicli for annidati e come collassarli tramite clausola `collapse` (clausola)

Se ci sono cicli `for` annidati, spesso è sufficiente parallelizzare solamente il ciclo **più esterno**.

Quando però il ciclo più esterno ha poche iterazioni in proporzione al numero di threads utilizzati, e si prova allora ad parallelizzare il ciclo interno, è possibile che l'efficienza rimanga comunque bassa.

```
a();
for (int i = 0; i < 3; ++i) {
    #pragma omp parallel for
    for (int j = 0; j < 6; ++j) {
        c(i, j);
    }
}
z();
```



La soluzione è far **collassare** il tutto in un **singolo ciclo**, ciò si può fare manualmente:

```
a();
#pragma omp parallel for
for (int ij = 0; ij < 3*6; ++ij) {
    c(ij / 6, ij % 6);
}
z();
```



o farlo fare ad OpenMP tramite la clausola `collapse` :


```
a();
#pragma omp parallel for
collapse(2)
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 6; ++j)
        c(i, j);
}
z();
```



OpenMP: Pianificazione dei cicli (Scheduling Loops)

Normalmente in OpenMP, quando si parallelizza un ciclo tramite `omp parallel for`, le varie iterazioni del ciclo vengono assegnate ai threads secondo un partizionamento di default che potrebbe **non** essere l'assegnazione più performante.

È quindi possibile impostare che tipo di assegnamento usare tra:

- `static`.
- `dynamic` o `guided`.
- `auto`.
- `runtime`.

```
# pragma ... schedule(type, chunksize)
```

Il `chunksize` indica quante iterazioni del ciclo assegnare alla volta a ogni thread.

OpenMP: Assegnamento statico (Static Loops Scheduling)

In questo tipo di assegnamento, le iterazioni vengono assegnate ai threads **prima** che il ciclo inizi l'esecuzione.

Viene fatto in modo che ad ogni thread venga assegnato più o meno la stessa quantità di iterazioni.

Ma è possibile vi siano iterazioni che richiedono più lavoro o tempo per finire, quindi

può causare **carichi di lavoro non equi**.

```
# pragma ... schedule(static, chunksize)

// E.g.
// Usando schedule(static, 4) con 12 iterazioni e 3 threads:
// Thread 0 ottiene: 0, 1, 2, 3
// Thread 1 ottiene: 4, 5, 6, 7
// Thread 3 ottiene: 8, 9, 10, 11

// E.g.
// Usando schedule(static, 1) con 12 iterazioni e 3 threads:
// Thread 0 ottiene: 0, 3, 6, 9
// Thread 1 ottiene: 1, 4, 7, 10
// Thread 3 ottiene: 2, 5, 8, 11
```

OpenMP: Assegnamento dinamico o guidato (Dynamic/Guided Loops Scheduling)

In questi tipi di assegnamento, le iterazioni vengono assegnate ai threads **mentre** il ciclo viene eseguito.

OpenMP: Assegnamento dinamico

In questo tipo di assegnamento, quando un thread finisce un chunk di iterazioni a lui assegnate, va a **richiedere un nuovo chunk** di iterazioni tra quelle rimaste.

È una strategia ideale per iterazioni **non bilanciate**, dove i thread che finiscono prima possono semplicemente andare a richiedere più lavoro. Tuttavia queste richieste introducono una latenza.

OpenMP: Assegnamento guidato

In questo tipo di assegnamento, quando un thread finisce chunk di iterazioni a lui assegnate, va a richiedere un nuovo chunk di iterazioni.

Rispetto all'assegnamento dinamico, la dimensione dei chunk non è fisso, ma diminuisce man mano che vengono completati.

Quindi i chunk hanno dimensione variabile $dim_chunks = \frac{num_iterations}{num_threads}$ dove

`num_iterations` diventerà il numero di iterazioni rimanenti dopo che un chunk di

iterazioni viene dato.

È una strategia che cerca di avere la bassa latenza dell'assegnamento statico e il bilanciamento del carico di lavoro dell'assegnamento dinamico.

OpenMP: Assegnamento automatico

Il compilatore e o il sistema determinano il tipo di assegnamento da usare.

OpenMP: Assegnamento a tempo di esecuzione (Runtime Loops Scheduling)

In questo tipo di assegnamento, l'assegnamento viene determinato a tempo di esecuzione.

Il sistema usa la **variabile d'ambiente** `OMP_SCHEDULE` per determinare a tempo di esecuzione come dovrebbe assegnare le iterazioni del ciclo.

Tale variabile può essere `static`, `dynamic` o `guided`, più il valore del `chunksize`.

```
$export OMP_SCHEDULE="static,1"
```

OpenMP: Direttive per la sincronizzazione

OpenMP: Master/Single (direttive)

Entrambe le direttive `master` e `single` **forzano l'esecuzione** del blocco di codice parallelo su un singolo thread.

La direttiva `single` implicitamente inserisce una **barriera** alla fine del blocco.

La direttiva `master` garantisce l'esecuzione del blocco sul thread master.

OpenMP: barrier (direttiva)

Serve per bloccare i thread del team che raggiungono la barriera, e li sblocca solamente quando **tutti** i thread del team raggiungono tale punto.

OpenMP: sections/section (direttive)

Servono per poter eseguire sezioni di codice **completamente indipendenti** in modo concorrente.

Ogni sezione viene eseguito **una volta** da **un singolo** thread, e se ci sono più sezioni che threads, allora alcuni thread eseguiranno più sezioni.

Siccome queste sezioni di codice vengono eseguite in modo concorrente, è necessario che siano completamente indipendenti.

Alla fine del blocco di codice segnato dalla direttiva `sections` implicitamente viene inserita una **barriera**. Tale barriera non viene inserita se esplicitamente specificato tramite una clausola `nowait`.

```
# pragma omp parallel sections
{
    #pragma omp section
    {
        ...
    }
    # pragma omp section
    {
        ...
    }
}
```

OpenMP: ordered (direttiva)

Serve all'interno di `omp parallel for` per **assicurare** che l'esecuzione di un blocco di codice indicato viene fatto in ordine **sequenziale**.

OpenMP: Gestione dei thread OpenMP quando generati da processi MPI

MPI assegna ad ogni processo un core, e quando un processo MPI genera un team di threads OpenMP, questi threads vengono **tutti** eseguiti sullo stesso core sulla quale gira il processo, portando possibilmente ad una velocità ridotta di esecuzione del programma.

Per sistemare questo problema, serve usare il *flag* `--bind-to-none`.

Dipendenze dei dati (Data Dependencies)

Generalmente OpenMP **non riesce** a parallelizzare correttamente un ciclo in cui il risultato di uno o più iterazioni dipende da altre iterazioni. Questo in quanto il compilatore OpenMP non controlla se ci sono dipendenze tra le iterazioni di un ciclo parallelizzato tramite la direttiva `parallel for`.

Quando almeno due operazioni operano sulla stessa zona di memoria, dove necessariamente uno di queste è un'operazione di **scrittura**, e sono divise in più iterazioni del ciclo, si viene a creare una **dipendenza trasportata dal ciclo** (Loop-carried dependence).

Esistono vari tipi di dipendenze:

- Flow dependence (RAW, or Read-After-Write).
- Anti-Flow dependence (WAR, or Write-After-Read).
- Output dependence (WAW, or Write-After-Write).

Flow dependence (Data Dependencies)

Accade quando si legge da una zona di memoria dopo che viene scritto sulla stessa zona di memoria.

```
x = 10; // Write x
y = 2 * x; // Read x
```

Anti-Flow dependence (Data Dependencies)

Accade quando si scrive su una zona di memoria dopo che viene letta la stessa zona di memoria.

```
y = x + 1; // Read x
x++; // Write x
```

Output dependence (Data Dependencies)

Accade quando si scrive su una zona di memoria dopo che viene scritta sulla stessa zona di memoria.

```
x = 10; // Write x
x = x + 10; // Write x
```

Risoluzione di dipendenze dei dati

Six primary techniques are used to resolve flow dependencies (Read-After-Write or RAW), alongside specific strategies for handling anti-dependences and output dependencies.

Here is a breakdown of every technique used to fix data dependencies:

1. Reduction and Induction Variable Fixes

- **Induction Variables:** An induction variable gets increased or decreased by a constant amount during each loop iteration, which causes a dependency between steps. You can remove this dependency by rewriting the variable as an affine function of the loop index (for example, changing `v = v + step` to calculate directly via `v = start + i * step`), which allows each iteration to calculate its value independently.
- **Reduction Variables:** When a variable accumulates a total result (like a sum), it creates a dependency. This is fixed by using the OpenMP `reduction` clause (e.g., `reduction(+:sum)`). The runtime creates a private, uninitialized copy of the variable for each thread, applies the operations locally, and then atomically combines all private copies into the global shared variable at the end of the region.

2. Loop Skewing

Loop skewing involves rearranging the statements within the loop body so that the values consumed during an iteration were generated during that exact same iteration. By manually unrolling the loop to observe the repetition pattern, you can "skew" the loop bounds. For example, if statement 1 relies on the previous iteration's statement 2, you can execute the very first instance of statement 1 outside the loop, shift the loop index, and execute them safely in parallel.

3. Fissioning

Fissioning means **breaking a single loop apart into multiple separate loops**: one that contains the problematic sequential dependencies, and another that contains the safe, parallelizable work. This ensures that the parallelizable portion of the work can still be accelerated using OpenMP without being held back by the sequential statements.

4. Refactoring (Wavefront Execution)

Refactoring involves rewriting the loops to expose hidden parallelism by analyzing the **Iteration Space Dependency Graph (ISDG)**. If the ISDG shows that dependencies flow strictly down and to the right (like a grid), there are no dependencies between nodes on the same diagonal. You can refactor the code to execute in "waves" along these diagonals, changing the loop variables so the outer loop iterates over the waves, and the inner loop processes the independent diagonal sets in parallel.

5. Partial Parallelization

By analyzing the Iteration Space Dependency Graph (ISDG), you may find that while an outer loop carries a dependency, an inner nested loop does not (e.g., there are no dependency edges between nodes on the same row). In these cases, you can partially parallelize the workload by applying the OpenMP directives exclusively to the independent inner loop.

6. Algorithm Change

If the code's dependencies cannot be removed through restructuring, **switching the underlying mathematical algorithm** may be the only answer. For example, the standard iterative method for calculating the Fibonacci sequence relies heavily on the previous two iterations; however, switching to Binet's formula allows you to calculate any Fibonacci number completely independently, making it trivially parallelizable.

Fixing Other Dependency Types

- **Removing Anti-Dependences (Write-After-Read / WAR):** This occurs when a loop writes to a variable that a future iteration needs to read. The simplest solution is to **make a copy of the array or variable** before you start modifying it. The threads can then safely read from the unmodified copy while writing to the original array. However, this requires careful consideration of the space and time trade-offs.
- **Removing Output Dependences (Write-After-Write / WAW):** This occurs when multiple iterations write to the same memory location, and you need to guarantee that the final value stored is from the logically "last" iteration. This is resolved by using the OpenMP `lastprivate` clause, which ensures that at the end of the parallel execution, the variable is updated with the value computed in the final sequential iteration of the loop.

Caching

Esistono due tipi di **località**:

- Località spaziale, quando si accede ad una zona di memoria vicina.
- Località temporale, quando si accede ad una zona di memoria frequentemente acceduta.

I dati vengono trasferiti dalla memoria alla cache **in blocchi** (o righe), cioè quando si trasferisce un valore di un vettore, possibilmente anche i valori nel vettore vicini vengono trasferiti.

Caching: Consistenza dei dati tra cache e memoria

Quando la CPU scrive dati nella cache, i valori scritti potrebbero essere **inconsistenti** rispetto ai valori presenti nella memoria principale.

Una cache **write-through** gestisce questi casi:

- Aggiornando i dati nella memoria principale nel momento in cui vengono scritti nella cache.

Una cache **write-back** invece gestisce questi casi:

- Marcando i dati scritti nella cache come **sporchi**.
- E aggiornando tali dati sulla memoria principale quando questi vengono rimpiazzati

nella cache da nuovi dati.

Caching: Coerenza dei dati tra varie cache di multipli cores

Dato che i programmatori non hanno un controllo diretto su quando vengono aggiornate le cache, si presenta un problema quando più cores contengono una copia della stessa variabile condivisa.

Per mantenere la coerenza di queste variabili condivise tra multipli core, esistono dei meccanismi hardware progettati appositamente.

Caching: Coerenza della cache tramite Snooping

In questa architettura, tutti i core **condividono un bus** di comunicazione comune.

Qualsiasi segnale trasmesso su questo bus può essere notato da tutti i core connessi, e quando un core aggiorna una variabile condivisa nella sua cache locale, trasmette queste informazioni sul bus.

Gli altri core **monitorano continuamente** questo bus e se un core rileva un aggiornamento per una variabile che attualmente contiene, contrassegna la sua copia locale nella cache come non valida.

Tuttavia, questo approccio è raramente utilizzato perché la trasmissione diventa troppo costosa e non scalabile su architetture con un numero elevato di core.

Caching: Coerenza della cache tramite Directory

Questo approccio utilizza una struttura dati dedicata chiamata **Directory** per mantenere traccia dello stato di ogni **linea di cache**.

La directory mantiene un **elenco** di quali core attualmente possiedono una copia di quella specifica linea.

Quando una variabile viene aggiornata, il sistema consulta la directory e invia esplicitamente **segnali di invalidazione** solo ai core specifici che effettivamente possiedono una copia di quella variabile.

Caching: False Sharing

(Leggere anche sopra - Coerenza della cache tramite Directory)

I dati contenuti nella memoria principale vengono copiati nella cache **in righe**, dove ognuna di queste linee può contenere multiple variabili.

Quando un dato (variabile) viene **invalidato**, **tutta la riga** che contiene tale dato viene invalidata, e di conseguenza ogni lettura di altre variabili sulla stessa riga, anche se corrette, vengono invalidate.

Per risolvere questo problema è possibile:

- Aggiungere un **padding** ai dati copiati in modo che variabili indipendenti siano su righe diverse.
- Usare variabili locali per raggruppare il risultato computato e solamente alla fine scrivere sul dato in cache.
- Cambiare l'assegnamento dei dati rispetto ai thread usati, assicurando che ad ogni thread venga assegnato una riga di cache non facilmente influenzabile da altri thread.

Effettuando il padding tuttavia causa un'efficienza ridotta della cache e spreca memoria.

Organizzazione della memoria (Memory Organization)

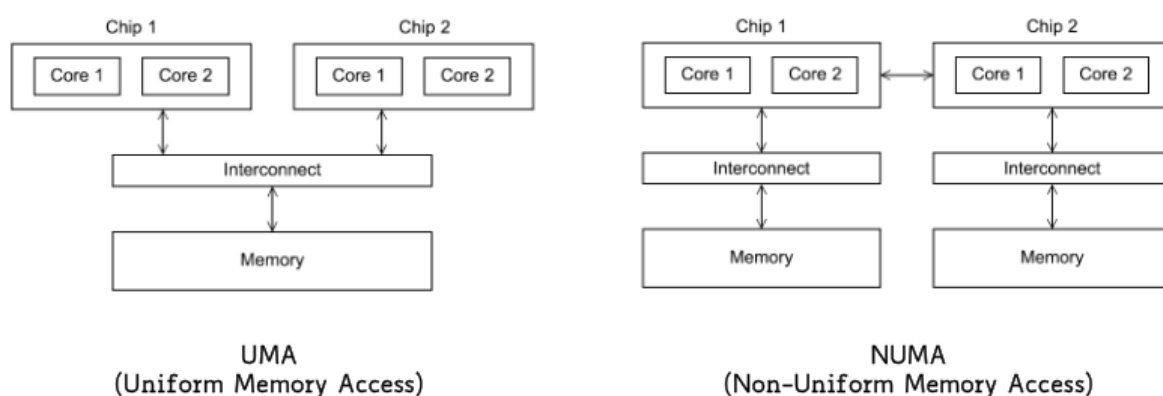
In multiprocessor computer systems, memory organization typically falls into two primary shared-memory architectures: **Uniform Memory Access (UMA)** and **Non-Uniform Memory Access (NUMA)**. While both models provide a single, shared address space where all memory is directly accessible by any processor, they differ fundamentally in their physical layout and access speeds.

Uniform Memory Access (UMA) In a UMA architecture, the time it takes for a processor to access memory is completely independent of which processor is making the request or where the data physically resides. Multiple CPUs share a central pool of global memory equally. Because all processors sit at the same "distance" from the memory, they all experience the exact same latency and bandwidth.

Non-Uniform Memory Access (NUMA) As systems scale to higher core counts, UMA becomes a bottleneck. Modern high-performance computing (HPC) nodes typically use NUMA architectures, where the physical memory is divided and distributed among

multiple nodes, with each node having its own processors and a directly attached memory pool.

- **Local vs. Remote Access:** While the entire memory space is still transparently accessible to all processors on the system, the access time varies wildly. A processor can access its own **local memory** much faster—with higher bandwidth and lower latency—than it can access **remote memory** (memory that is local to another processor).
- **The Interconnect Penalty:** Accessing remote memory requires the data request to cross a socket-to-socket interconnect network. If a program forces cores to constantly access remote memory, the overall memory bandwidth is severely degraded and becomes dominated by the limited speed of the inter-socket links.



GPU

Le GPU sono progettate per poter effettuare trasferimenti di grandi quantità effettive di dati durante le operazioni in un certo periodo di tempo (rendimento, numero totale di operazioni completate al secondo) (throughput). Per ottenere ciò posseggono molte più unità di calcolo (Arithmetic Logic Unit) rispetto alle CPU.

Inoltre:

- Le unità di controllo sono più semplici rispetto alle CPU.
- La cache ha dimensioni ridotte rispetto alle CPU.

Le unità di calcolo sono efficienti, ma presentano una latenza molto alta, perciò richiedono molti thread per tollerare questa latenza.

GPU: Architettura di una GPU che può eseguire CUDA

L'architettura fisica di una GPU CUDA compatibile è organizzata come un *array* di **Streaming Multiprocessors** (SM), queste sono le unità funzionali di computazione più grandi all'interno dell'architettura.

Due o più SMs formano un *building block*.

All'interno di ogni SM sono contenuti gli **Streaming processor** (SP), chiamati anche come **CUDA cores**. Queste sono le unità responsabili delle operazioni aritmetiche tra numeri interi e in virgola mobile.

CUDA

Compute Unified Device Architecture (CUDA) è un modello di programmazione, che permette di programmare su GPU NVIDIA.

Il modello di esecuzione CUDA involve l'esecuzione concorrente sia del *host* (CPU) che del *device* (GPU), dove:

- Le operazioni di I/O vengono eseguite dal host.
- Le operazioni di computazione di dati vengono eseguiti dal device.

CUDA: Organizzazione logica dei thread

I thread in CUDA sono organizzati logicamente in una **struttura a sei dimensioni (al massimo)**:

- 3 dimensioni per le griglie (*grid*).
- 3 dimensioni per i blocchi (*block*).

Una griglia è composta da blocchi di threads, e attraverso una serie di variabili interne, ogni thread conosce la sua posizione all'interno della struttura.

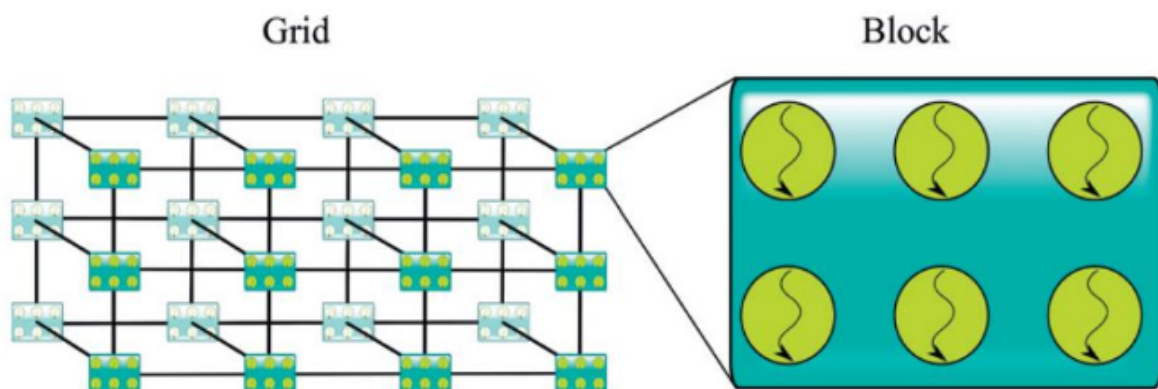
```
dim3 blockDim; // Struttura che contiene i 3 valori delle dimensioni dei blocchi
i

dim3 gridDim; // Struttura che contiene i 3 valori delle dimensioni delle griglie
ie

uint3 threadIdx; // Struttura che contiene i 3 valori della posizione del thread
d rispetto al blocco

uint3 blockIdx; // Struttura che contiene i 3 valori della posizione del blocco
rispetto alla griglia
```

La dimensione di un blocco non può superare 1024 threads. (Attualmente)



CUDA: Capacità di calcolo (SM version)

La dimensione dei blocchi e delle griglie è determinato dalla **capacità**, che determina anche di cosa è capace ogni generazione di GPU NVIDIA.

Tale capacità viene identificata attraverso un numero (version number), o chiamato *SM version*, che identifica le caratteristiche (*features*) supportate dall'hardware della GPU.

CUDA: Come ottenere la posizione di un thread

Dato un thread situato all'interno di una griglia di blocchi, per ottenere la sua posizione globale:

```
int block_position = blockIdx.x; // Posizione del blocco contenente il thread d
a ottenere rispetto a tutti i blocchi nella griglia

int block_dimension = blockDim.x; // Dimensione fissa per ogni blocco

int thread_position = threadIdx.x; // Posizione del thread nel suo blocco rispe
tto a tutti gli altri thread nel suo blocco

int global_thread_position_x = block_position * block_dimension + thread_positi
on
```

CUDA: Thread scheduling

In una GPU, ogni thread viene eseguito su un *SP*.

Un insieme di CUDA cores nella stessa SM **condividono l'unità di controllo**, ovvero questi threads eseguono simultaneamente tutti la stessa istruzione.

CUDA: Warps

Dato un blocco di threads, questi sono eseguiti in gruppi chiamati **warps**:

- I threads vengono divisi in warps in base al loro `threadId`.

In ogni ciclo di clock, all'interno di un warp, **tutti i thread** del warp eseguono la stessa istruzione assegnata.

È possibile avere multipli **warp scheduler** presenti su un *SM*, in modo tale che multipli warps possano essere eseguiti allo stesso tempo.

CUDA: Divergenza dei thread di un warp (Warp Divergence)

Siccome tutti i thread all'interno di un warp eseguono la stessa istruzione secondo il modello SIMD ([002 - Rielaborato#Sistemi paralleli Con singola unità di controllo (SIMD)|Single-Instruction Multiple-Data]), quando si presenta una divergenza di esecuzione causata per esempio da una condizione `if/else`, l'hardware va a serializzare i differenti rami di esecuzione:

- I thread che soddisfano la condizione vanno ad eseguire completamente un ramo.

- I thread che **non** soddisfano la condizione vengono **bloccati**.
- Quando i thread che soddisfano la condizione **finiscono** l'esecuzione del loro ramo, gli altri thread che non soddisfano la condizione vanno ad eseguire il loro ramo, e viceversa i rimanenti thread vengono bloccati.
- L'esecuzione sequenziale continua per ogni ramo divergente, finché non si ricongiungono tutti i rami, e a quel punto, tutti i thread riprendono ad eseguire le stesse istruzioni in modo sincronizzato.

Nel caso peggiore, può essere che ogni thread all'interno del warp esegua singolarmente un ramo in modo sequenziale, **diminuendo il throughput** del warp fino a un $\frac{1}{32}$ (se 32 threads per warp).

CUDA: Context Switching

Generalmente un *SM* possiede più warp residenti di quanti ne può eseguire in modo concorrente, e data la possibilità di poter scambiare tra warps in esecuzione e quelli residenti in modo fluido (*seamlessly*), quando un'istruzione da eseguire su un warp ha bisogno di attendere dei risultati computati da un'altra operazione già iniziata, tale warp viene scambiato e non verrà scelto per l'esecuzione finché non otterrà tutti i dati necessari.

Questo meccanismo di scambio del contesto dei warp serve per **nascondere la latenza** delle operazioni sulle GPU.

Questa caratteristica di poter **tollerare** operazioni con latenze alte **è la ragione principale** per cui le GPU **non** hanno bisogno di dedicare tanto spazio per **memoria cache** e **meccanismi di predizione dei rami** rispetto alle CPU.

CUDA: Come scrivere un programma CUDA

Per poter usare la GPU con CUDA, bisogna definire una funzione (*kernel*) dove al suo interno bisogna **specificare l'organizzazione dei thread** in blocchi e griglie.

```
int x, y, z;
dim3 block(x, y, z);
dim3 grid(x, y, z);
kernel_function<<<grid, block>>>();
```

CUDA: Decoratori delle funzioni

Durante la definizione di una qualsiasi funzione, serve inserire un decoratore. Ciò serve per specificare quale risorsa tra CPU e o GPU potrà eseguire quella funzione:

- Una funzione `__global__` può essere chiamata sia dalla CPU che dalla GPU e può essere eseguita sia dalla CPU che dalla GPU.
- Una funzione `__device__` può essere chiamata solo dall'interno di una funzione *kernel*, ovvero dalla GPU, e può essere eseguita solamente dalla GPU.
- Una funzione `__host__` può essere eseguita solamente dalla CPU, e generalmente viene omesso.
 - Viene specificato solo se usato insieme a `__device__` per indicare che tale funzione può essere eseguito su entrambi.

CUDA: Accesso alla memoria e come gestirla

Dato una zona di memoria allocata nella DRAM e dei dati al loro interno, questi **non** sono visibili dalla GPU e viceversa.

```
int* data = new int[N]; // Array nella DRAM

kernel_function<<<grid, block>>>(data, N); // Non funziona
```

Serve quindi esplicitamente **copiare** tali dati dall'host alla GPU e viceversa.

Per copiare dati dall'host alla GPU serve prima allocare zone di memoria sulla VRAM della GPU tramite la funzione `cudaMalloc`.

```
cudaError_t cudaMalloc(void** d_ptr, size_t size);
```

Per copiare i dati nella nuova zona di memoria allocata serve usare `cudaMemcpy`.


```
cudaError_t cudaMemcpy(void* dest, const void* src, size_t size, cudaMemcpyKind
direction_of_copy);

// direction_of_copy = cudaMemcpyHostToHost = 0
// direction_of_copy = cudaMemcpyHostToDevice = 1
// direction_of_copy = cudaMemcpyDeviceToHost = 2
// direction_of_copy = cudaMemcpyDeviceToDevice = 3 // Per configurazioni con mu
ltiple GPU
// direction_of_copy = cudaMemcpyDefault = 4 // Usato quando la RAM è unificata
(CPU e GPU condividono la RAM)
```

E infine per liberare la zona di memoria a fine programma con `cudaFree` .

```
cudaError_t cudaFree(void* d_Ptr);
```

CUDA: Esempio kernel somma di due vettori

```
__global__ void add_vector_kernel(float* d_A, float* d_B, float* d_C, int n) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;

    // Solo se il numero del thread è contenuto nella dimensione dei vettori
    // Altrimenti il thread andrebbe fuori dal vettore
    if (i < n) {
        d_C[i] = d_A[i] + d_B[i];
    }
}

void add_vector(float* h_A, float* h_B, float* h_C, int vector_elements) {
    int size = vector_elements * sizeof(float);
    float* d_A;
    float* d_B;
    float* d_C;

    cudaMalloc((void**) &d_A, size);
    cudaMalloc((void**) &d_B, size);
    cudaMalloc((void**) &d_C, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // kernel invocation
    add_vector_kernel<<<ceil(vector_elements/256.0), 256>>>(d_A, d_B, d_C,
vector_elements);

    // La dimensione della griglia è uguale al numero di blocchi necessari
    // per coprire il numero di elementi dei vettori
    // Ogni blocco è composto da 256 threads
    // Ogni thread esegue una singola addizione tra gli elementi allo stesso
    // index nei due vettori

    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
}
```

CUDA: Occupancy (Occupazione/Capienza)

Il rapporto tra il numero di warp residenti e il numero massimo possibile di warp residenti viene definito come **occupancy**: $occupancy = \frac{resident_warps}{maximum_warps}$ Più questo valore è vicino a 1, più sono le possibilità per la GPU di nascondere le latenze grazie allo scambio dei thread.

È possibile aumentare l'occupancy riducendo il numero di registri necessari alla funzione kernel, evitando di avere troppe variabili temporanee...

CUDA: Stima della performance ottenibile (Performance estimation)

Performance estimation is a critical process used to determine whether a program is successfully saturating the computational capabilities of the GPU hardware. Based on the slides and corroborated by multiple other architectural guides, here are the core concepts used to estimate and analyze performance:

1. FLOP/s (Floating-Point Operations Per Second) The fundamental metric for measuring computational performance is **FLOP/s**, which indicates the number of floating-point operations the hardware executes per second. When citing this metric, it is essential to specify the data precision being used (such as 16-bit, 32-bit, or 64-bit floating-point), as hardware execution rates vary wildly depending on the precision. Modern top-tier supercomputers can achieve up to 1 ExaFLOP/s (10^{18} FLOP/s).

2. The Memory Bandwidth Bottleneck Achieving a GPU's theoretical peak FLOP/s rate is rare because performance is frequently limited by how fast data can be fed from global memory to the execution cores. For example, if a GPU has a global memory bandwidth of 200 GB/s and uses 4-byte operands, it can load a maximum of 50 billion (50G) operands per second ($200 \text{ GB/s} / 4 \text{ bytes} = 50 \text{ G operands/s}$). If your algorithm performs exactly one floating-point operation (like a simple addition) for each operand it loads, the maximum possible performance is capped at 50 GFLOP/s.

3. Memory-Bound vs. Compute-Bound If the GPU in the previous example has a theoretical peak compute capacity of 1,500 GFLOP/s (1.5 TFLOP/s), running at 50 GFLOP/s means the application is utilizing only **3.3%** of the processor's actual capabilities.

- **Memory-Bound:** In this scenario, the application is memory-bound because data movement to and from memory limits the performance, rather than the arithmetic execution units. Simple kernels like vector addition or layer normalization naturally fall into this category.

- **Compute-Bound:** Conversely, compute-bound kernels perform enough arithmetic operations per byte accessed to fully saturate the GPU's floating-point units (ALUs), such as dense matrix multiplications.

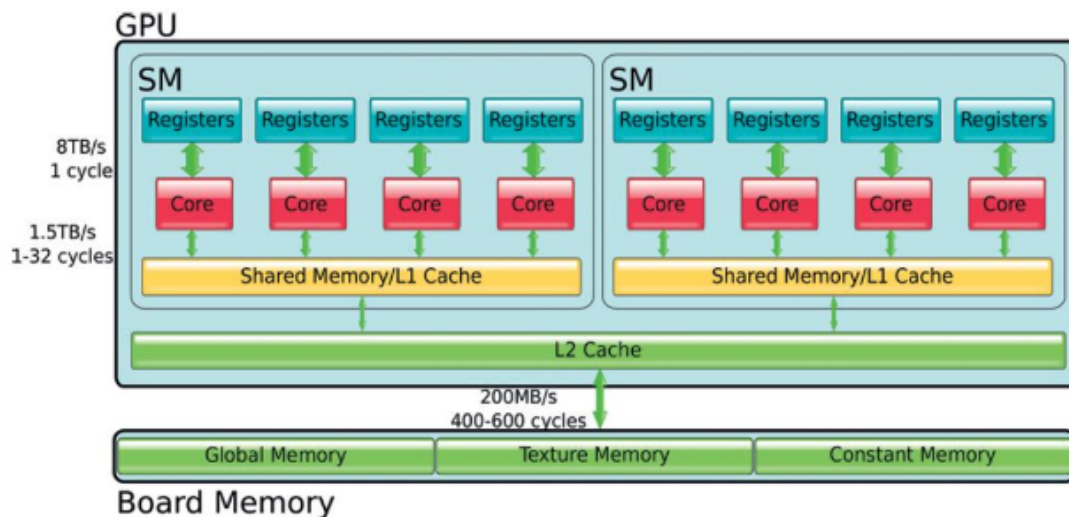
4. Arithmetic Intensity (Operational Intensity) To determine how to push a memory-bound application closer to peak performance, developers analyze the **compute-to-global-memory-access ratio**, widely known as **arithmetic intensity**. This ratio represents the number of mathematical operations performed for every byte (or operand) fetched from memory.

Using the 1.5 TFLOP/s GPU example, to reach peak performance while loading 50G operands per second, the algorithm requires an arithmetic intensity of at least **30** ($1.5 \text{ TFLOP/s} / 50\text{G operands/s} = 30$). This means the code must perform 30 math operations on every single operand it fetches from memory, reusing data extensively to avoid bandwidth limits.

Cross-Verification with Other Sources (The Roofline Model) Other performance tuning guides confirm these exact principles using a concept known as the **Roofline Model**. The model dictates that an application's performance is mathematically capped by the formula: `performance ≤ min(compute_peak, arithmetic_intensity × bandwidth)`. For instance, on a modern NVIDIA A100 GPU with 19.5 TFLOPS (FP32) peak compute and 1.6 TB/s of bandwidth, a kernel must perform roughly 12 floating-point operations for every single byte loaded just to saturate the compute units.

Understanding and optimizing arithmetic intensity is increasingly critical because the hardware industry is experiencing a trend where computational throughput is growing at a much faster rate than memory bandwidth.

CUDA: Tipi di memoria



All'interno delle GPU sono presenti vari tipi di memoria, alcune sono all'interno delle SM mentre altre sono all'esterno delle SM:

- I **registri** mantengono le variabili locali.
- La **memoria condivisa** è una memoria interna (on-chip memory) molto veloce che mantiene dati frequentemente usati. Inoltre viene usato per scambiare dati tra i vari core all'interno di una stessa SM.
- La **memoria di livello 1 e 2 (cache)** sono nascoste al programmatore (non gestibili).
- La **memoria globale** è la parte principale della memoria esterna (off-chip memory), molto capiente ma relativamente lenta. Questa è l'unica memoria **accessibile dall'host** tramite funzioni CUDA.
- La memoria specializzata per **texture e superfici**, gestita da hardware speciali che permettono l'uso di operatori di filtraggio e interpolazione.
- La **memoria per le variabili costanti**, che permette di eseguire il **broadcasting** di singoli valori a tutti i thread di un warp. Questa memoria non è più molto usata in quanto le GPU hanno ormai una memoria cache.

CUDA: Registri (Registers)

Tutti i thread che risiedono nel core si **dividono** i suoi registri, che vengono usati per memorizzare le variabili locali usate dai threads.

Il numero di registri che può essere assegnato per thread **varia in base** alla capacità di calcolo della GPU.

E se il numero di registri necessario al thread **supera** il limite, le variabili locali in più

del thread verranno allocate o nella cache L1 (on-chip) oppure nella memoria globale (off-chip), tale allocazione viene decisa dal compilatore.

CUDA: Memoria condivisa (Shared Memory)

È una memoria interna (on-chip) alla *SM* che permette ai core di una stessa *SM* di **condividere dati**.

È possibile specificare quali dati contenere nella memoria condivisa invece che nella memoria globale usando `__shared__`.

Generalmente la memoria condivisa è affiancata alla **cache di livello 1**, tuttavia la prima è gestibile dal programmatore, mentre la seconda è gestita automaticamente dall'hardware.

```
__shared__ int vector[10]; // Esempio
```

La memoria condivisa è suddivisa in **banchi** (banks):

- Ogni banco può servire **un singolo accesso** per ciclo.

CUDA: Conflitti tra blocchi (bank) della memoria condivisa

Multipli accessi a un singolo banco richiederanno molti cicli.

Quindi i threads all'interno di un warp dovrebbero evitare di accedere allo stesso momento a un banco di memoria condivisa uguale.

Inoltre, se tutti i threads all'interno di un warp accedono alla stessa zona di memoria, l'hardware eseguirà una **lettura in broadcast**, oppure se multipli threads ma non tutti, allora una **lettura in multicast**.

CUDA: Come prevenire pericoli di RAW, WAR o WAW (Usando __syncthreads)

`__syncthreads__` è una **barriera** usata per prevenire le dipendenze dei dati come RAW, WAR e WAW.

Tutti i threads per poter continuare la loro esecuzione devono prima raggiungere la barriera.

È necessario quando si opera su un vettore **in-place**, ovvero si usano e modificano i dati contenuti all'interno del vettore senza usare una copia.

```
... // Computa il risultato ottenuto usando valori del vettore in una variabile
    locale

__syncthreads(); // Tutti i threads hanno il loro risultato in una variabile lo
    cale

... // Scrivi il risultato nel vettore
```

CUDA: Memoria per le variabili costanti (Constant Memory)

La memoria per le costanti serve per memorizzare valori costanti, queste sono *cached* e possono essere inviate a tutti i thread di un warp tramite *broadcasting*.

La memoria per le costanti **non** può essere allocata **dinamicamente**.

```
__constant__ float vector[10];

cudaMemcpyToSymbol(vector, hostData, sizeof(float)*10);
```

CUDA: Moltiplicazione di matrici

TBD

Memoria virtuale (Virtual memory)

Poiché la memoria principale è **fisicamente limitata**, il sistema operativo usa il meccanismo della **memoria virtuale**, che associa ad uno stesso indirizzo fisico, multipli indirizzi virtuali.

Questi indirizzi virtuali sono raggruppati in **pagine** e vengono spostati dentro e fuori dalla memoria fisica.

Gli indirizzi virtuali necessitano di essere **tradotti** in indirizzi fisici quando si accede ai dati puntati.

Quando la memoria fisica si riempie, è possibile spostare le pagine di memoria virtuale fuori dalla memoria fisica per ottenere dello spazio libero.

CUDA: Trasferimento di dati usando DMA

Per spostare dati in modo efficiente tra la CPU e la GPU, `cudaMemcpy` utilizza dell'hardware speciale chiamato **Direct Memory Access** (DMA).

Quest'unità è progettata per trasferire grandi quantità di dati tramite l'**interconnessione del sistema** (PCIe bus solitamente).

Quindi **delegando le operazioni di trasferimento** dati al DMA, il sistema libera la CPU per eseguire altre operazioni in modo concorrente.

Siccome il DMA utilizza **indirizzi fisici**, si viene a creare un conflitto tra il funzionamento del DMA e il funzionamento della memoria virtuale:

- Il DMA **traduce gli indirizzi virtuali** e controlla che tutte le pagine richieste sono fisicamente presenti all'interno della memoria **all'inizio del trasferimento**.
- Ma esegue questa traduzione **una sola volta** per tutta la durata del trasferimento per poter avere massima efficienza e banda possibile.
- Il problema nasce quando il **sistema potrebbe spostare pagine** di indirizzi virtuali fuori dalla memoria fisica per inserirne altri dentro.
- Così il DMA leggerebbe dati sbagliati.

Per sistemare questo problema è possibile usare la **memoria fissata**.

CUDA: Memoria fissata (Pinned Memory)

La memoria fissata è una zona di memoria che **contiene pagine di indirizzi virtuali**, specificatamente **marcati** in modo che non possano essere spostati fuori dalla memoria fisica.

Quindi la memoria della CPU che viene utilizzata per operazioni di trasferimento dati dal DMA necessariamente devono essere allocate come memoria fissata.

Altrimenti, se i dati da trasferire stanno su una zona di memoria **non fissata**, sarà

necessario copiare tali dati su una zona di memoria fissata, e ciò comporta una **latenza aggiuntiva**.

Quindi, `cudaMemcpy()` è più veloce se i dati da copiare dall'host al device sono allocati in zone di memoria fissata, siccome non sono necessarie copie extra.

```
// Fissare memoria (host) usando funzioni non CUDA
malloc();
mlock();
...
munlock(); // De-allocazione
free();

// Fissare memoria (host) usando funzioni CUDA
cudaMallocHost();
...
cudaFreeHost(); // De-allocazione
```

CUDA: Memoria globale (Global Memory)

La memoria globale è la parte principale della memoria esterna (off-chip memory), molto capiente ma relativamente lenta. Questa è l'unica memoria **accessibile dall'host** tramite funzioni CUDA.

CUDA: Raggruppamento di zone della memoria globale (Global Memory Coalescing)

Quando un thread accede ad una zona di memoria, quello che effettivamente legge sono **una serie di zone consecutive** di memoria.

Quindi quando tutti i thread in un warp eseguono un'istruzione di lettura da memoria, l'hardware controlla se questi threads stanno cercando di accedere a zone consecutive di memoria, e in tal caso, **raggruppa tutti questi accessi in un singolo accesso**.

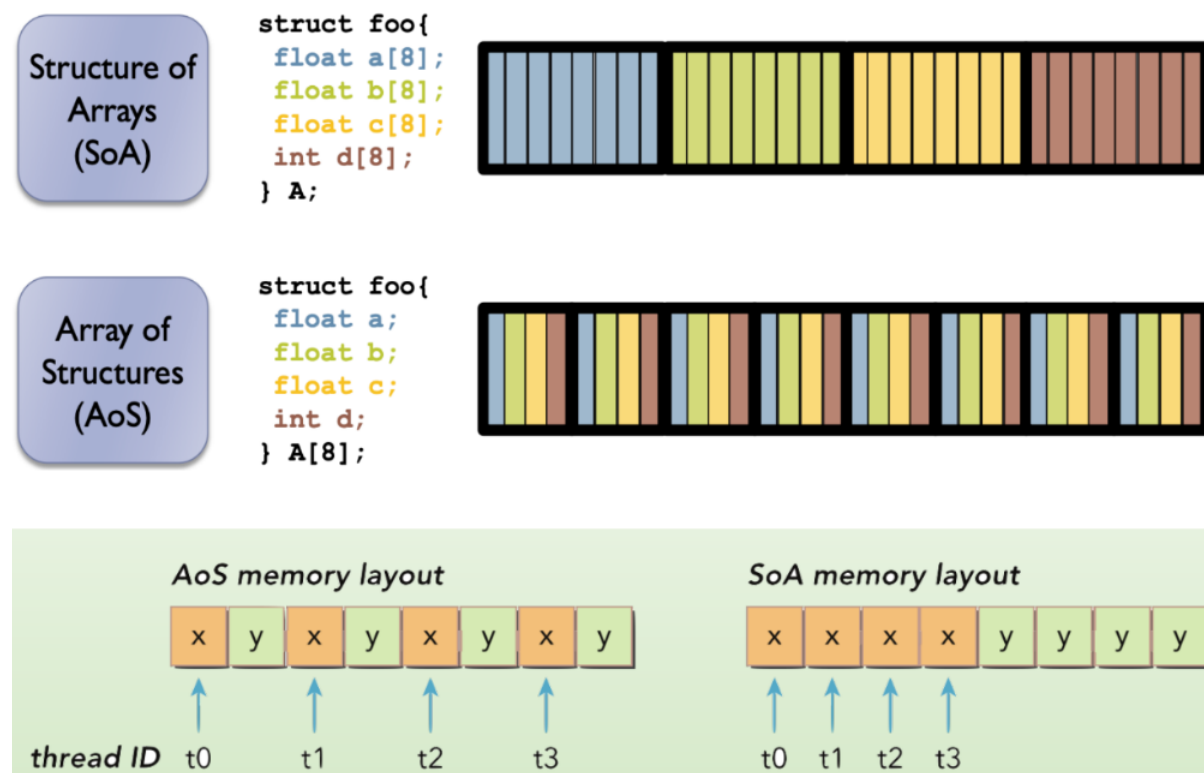
Organizzazione delle strutture dati per migliorare accessi raggruppati

Un array di structs o uno struct di arrays?

L'organizzazione della memoria per un array di structs **non è adatta** agli accessi di memoria raggruppati.

Mentre usando uno struct contenente gli array, si avrebbero tutti i dati necessari allineati in quanto dentro un array, adatto quindi agli accessi raggruppati.

Inoltre, gli struct potrebbero aver bisogno di **padding**, quindi in un array di struct, è possibile che ogni struct sia paddato, richiedendo quindi più memoria.



- Quando la cache L1 è disabilitata, le richieste di memoria bypassano L1 e vanno direttamente alla cache L2. La cache L2 opera con una granularità di segmento molto più fine, pari a 32 byte.

Se i thread in un warp richiedono parole di 4 byte sparse o non allineate, l'hardware **deve caricare più linee** da 128 byte per soddisfare la richiesta. Questo spreca un'enorme quantità di larghezza di banda del bus di memoria trasferendo dati circostanti che i thread **non utilizzeranno mai**, riducendo drasticamente l'utilizzo del bus.

Poiché i caricamenti senza cache recuperano i dati in blocchi più piccoli da 32 byte, attenuano la penalità derivante da modelli di accesso errati riducendo drasticamente il "recupero eccessivo".

In breve, disabilitare la cache L1 può prevenire un grande spreco di larghezza di banda.

CUDA + MPI

È possibile muovere dati tra più GPUs in base alla caratteristica della libreria MPI in uso:

- Se MPI è **GPU-aware** allora può direttamente accedere ai buffer delle GPUs, permettendo quindi di usare puntatori alla memoria della GPU come argomenti di funzioni MPI.
- Se MPI **non è** GPU-aware, allora i dati devono essere esplicitamente trasferiti prima dalla GPU all'host, eseguire la funzione MPI, e poi trasferire dall'host alla GPU.

Con librerie MPI GPU-aware, non c'è bisogno di copiare dati tra la memoria delle GPUs e dell'host. Ciò salva almeno un'operazione di copia dati per inviare e una per ricevere.

È utile anche per GPUs sullo stesso nodo.